

Reinforcement Learning

Chapter 3: Model-free RL

Ali Bereyhi

`ali.bereyhi@utoronto.ca`

Department of Electrical and Computer Engineering
University of Toronto

Fall 2026

Control versus Prediction

What we have done up to now is *prediction*

Prediction

*We are given by a *policy* and intend to evaluate it by *sampling**

But, in most applications we deal with a *control* problem

Control

*We are looking to move towards *optimal policy* while we are *sampling**

- + *But, we already talked about *GPI with sampling*! Didn't we?*
- *Well! What we did makes sense if we learn *offline*!*

GPI in Control Loop: Offline vs Online Approach

In *offline* RL, we *sample* environment first and find the *optimal policy* later

- 1 *Sample environment* with *sufficient* number of episodes
- 2 *Evaluate policies and improve them from the available dataset*
 - ↳ Go over the *dataset* over and over if needed

We are however looking for an *online* RL approach that *samples* environment and learns *optimal policy* gradually as it samples

- 1 Take a single *sample* from *environment*, e.g., a single reward-state pair or a terminating trajectory
- 2 *Estimate values and improve policy based on this single sample*
- 3 *Take a new sample . . .*

Direct GPI with Prediction

Recall a generic form of GPI with an evaluation approach X

$X_PolicyItr()$:

1: Initialize two random policies π and $\bar{\pi}$

2: **while** $\pi \neq \bar{\pi}$ **do**

3: $\hat{q}_\pi = X_QEval(\pi)$ and $\pi \leftarrow \bar{\pi}$

4: $\bar{\pi} = Greedy(\hat{q}_\pi)$

5: **end while**

and $Greedy(\hat{q}_\pi)$ is the basic improvement strategy

$Greedy(\hat{q}_\pi)$:

1: **for** $n = 1 : N$ **do**

2: Improve the by taking **deterministically** the **best action**

$$\bar{\pi}(a^m | s^n) = \begin{cases} 1 & m = \underset{m}{\operatorname{argmax}} \hat{q}_\pi(s^n, a^m) \\ 0 & m \neq \underset{m}{\operatorname{argmax}} \hat{q}_\pi(s^n, a^m) \end{cases}$$

3: **end for**

Direct GPI with Prediction

Recall a generic form of GPI with an evaluation approach X

```

X_PolicyItr():
1: Initiate two random policies  $\pi$  and  $\bar{\pi}$ 
2: while  $\pi \neq \bar{\pi}$  do
3:    $\hat{q}_\pi = X\_QEval(\pi)$  and  $\pi \leftarrow \bar{\pi}$ 
4:    $\bar{\pi} = Greedy(\hat{q}_\pi)$ 
5: end while
  
```

If we want our evaluation to be **accurate enough**: we need to keep playing **each policy** for a **large** number of episodes

- This is **extremely sample inefficient**
 - ↳ Imagine how many times we should **lose** the game to update our strategy!
 - ↳ We as human **do not really need** so many **losses**
- In many practical settings, it is really **cost-inefficient**
 - ↳ Amount of **losses** in each iteration to evaluate a policy is **not worth it!**

Online Control Loop via GPI

- + But, how can we do anything about this?
- Maybe we could improve the policy after *each update* of *action-values*

```
X_Control():
```

```
1: Initiate two random policies  $\pi$  and  $\bar{\pi}$ 
```

```
2: while  $\pi \neq \bar{\pi}$  do
```

```
3:    $\hat{q}_{\pi} = \text{X\_QUpdate}(\pi)$  and  $\pi \leftarrow \bar{\pi}$ 
```

```
4:    $\bar{\pi} = \text{Greedy}(\hat{q}_{\pi})$ 
```

```
5: end while
```

Here, $\text{X_QUpdate}(\pi)$ refers to one single update which is typically of the form

$$\hat{q}_{\pi}(S_t, A_t) \leftarrow \hat{q}_{\pi}(S_t, A_t) + \alpha(G - \hat{q}_{\pi}(S_t, A_t))$$

for some G computed according to *approach X*

First Try: Monte-Carlo Control Loop

We can build a Monte-Carlo control loop for **episodic environments**

MC_Control():

- 1: Initialize π and estimator as $\hat{q}_\pi(s, a) = 0$ for **all states** and **actions**
- 2: **for** episode = 1 : K or until π stops changing **do**
- 3: Initialize with a **random state-action pair** (S_0, A_0)
- 4: Act via $\pi = \text{Greedy}(\hat{q}_\pi)$
- 5: Sample a trajectory

$$S_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \dots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T: \text{terminal}$$

- 6: Initialize with $G = 0$
- 7: **for** $t = T - 1 : 0$ **do**
- 8: Update current return $G \leftarrow R_{t+1} + \gamma G$
- 9: Update $\hat{q}_\pi(S_t, A_t) \leftarrow \hat{q}_\pi(S_t, A_t) + \alpha(G - \hat{q}_\pi(S_t, A_t))$
- 10: **end for**
- 11: **end for**

Monte-Carlo Control: *Updating Action-Values*

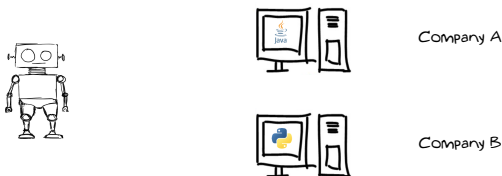
Comparing with `MC_QEval()`, there is only one difference, i.e.,

in-loop greedy improvement of the policy: line 4

- ① Estimate *action-values* over a *sample trajectory*
- ② *Improve* the policy using this estimate by *greedy approach*
- ③ *Sample* the next trajectory using the *improved policy*

-
- + Can we guarantee the convergence of this control loop?
 - In the current state, **not really!** Let's see an example!

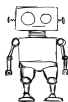
Example: Our Multi-armed Bandit



Recall our **first RL problem** in which the robot is to decide for a company

- ① it starts with a random decision **Company B**
 - ↳ Say it decides for Company B and receives **\$150 income**
 - ↳ Now, we have $\hat{q}_{\pi}(_, A) = 0$ and $\hat{q}_{\pi}(_, B) = 150$
- ② in the next episode, it would definitely chooses to work at **Company B**
 - ↳ Say it receives **\$250 income** this time
 - ↳ Now, we have $\hat{q}_{\pi}(_, A) = 0$ and $\hat{q}_{\pi}(_, B) = 200$
 - ⋮

Example: Our Multi-armed Bandit



Company A



Company B

The robot keeps working at **Company B!**

- + But, can we **guarantee** that Company A is not paying better?!
- Well! **Not really!** In fact, even if we had worked there for a single day or so, we could still not guarantee!

Greedy Improvement: *Lack of Exploration*

- + Why is this happening? Why it doesn't happen when we apply *direct GPI* via *Monte-Carlo*?
- In the latter, we do *exploration*; but now, we are only *exploiting*!

Downside of Greedy Improvement

In *greedy* improvement, we only *exploit* our knowledge, i.e.,

we always act *optimal* based on what we know *up to now*

We thus *lack exploration*, i.e.,

we remain unaware about *states* and *actions* that we have *not explored*

We may *never* get the chance to *explore* them!

Improving via ϵ -Greedy Improvement

A classical approach to handle this issue is to **improve** by ϵ -greedy approach

ϵ -Greedy Improvement

Choose a small $0 < \epsilon < 1$, and **improve** after each update of **action-values** by ϵ -greedy approach: at beginning of each episode

- ➡ with probability $1 - \epsilon$ act by the **improved** policy, i.e., **greedy**
- ➡ with probability ϵ act **randomly**, i.e., **explore the environment**

We did this in **Assignment 1** to render an **exploitation-exploration trade-off**

- with probability $1 - \epsilon$ we **exploit** our **improved** policy
- with probability ϵ we **explore** the environment

It's **hard** to find any improvement approach that can **beat** ϵ -greedy!

ϵ -Greedy Algorithm

We can algorithmically specify ϵ -greedy as

ϵ -Greedy($\hat{q}_\pi$):

1: **for** $n = 1 : N$ **do**

2: Take next step *randomly* as

$$\bar{\pi}(a^m | s^n) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{M} & m = \underset{m}{\operatorname{argmax}} \hat{q}_\pi(s^n, a^m) \\ \frac{\epsilon}{M} & m \neq \underset{m}{\operatorname{argmax}} \hat{q}_\pi(s^n, a^m) \end{cases}$$

3: **end for**

- + That seems to solve exploration problem! But, is there any *guarantee* that $\bar{\pi}$ is going to be a *better* policy?
- Yes! We can actually prove it!

ϵ -Greedy Algorithm

Let's assume we have policy π given after ϵ -greedy improvement, and we improved it again via the ϵ -greedy approach from its action-values: we can then write the value of new policy $\bar{\pi}$ as

$$\begin{aligned} v_{\bar{\pi}}(s) &= \sum_{m=1}^M \bar{\pi}(a^m|s) q_{\bar{\pi}}(s, a^m) \\ &= \underbrace{\frac{\epsilon}{M} \sum_{m=1}^M q_{\pi}(s, a^m)}_{\text{exploration}} + \underbrace{(1 - \epsilon) q_{\pi}(s, a^*)}_{\text{exploitation}} \end{aligned}$$

We know that for any non-negative $w_1, \dots, w_M$ that add up to one, we have

$$\sum_{m=1}^M w_m q_{\pi}(s, a^m) \leq q_{\pi}(s, a^*)$$

ϵ -Greedy Algorithm

Let's now define

$$w_m = \frac{\pi(a^m|s) - \epsilon/M}{1 - \epsilon}$$

We note that since π is an ϵ -greedy policy, we have $w_m \geq 0$ and

$$\sum_{m=1}^M w_m = \sum_{m=1}^M \frac{\pi(a^m|s) - \epsilon/M}{1 - \epsilon} = 1$$

So, we have

$$q_\pi(s, a^\star) \geq \sum_{m=1}^M \frac{\pi(a^m|s) - \epsilon/M}{1 - \epsilon} q_\pi(s, a^m)$$

ϵ -Greedy Algorithm

Now, let us replace this bound in the previous equation

$$\begin{aligned}
 v_{\bar{\pi}}(s) &= \frac{\epsilon}{M} \sum_{m=1}^M q_{\pi}(s, a^m) + (1 - \epsilon) q_{\pi}(s, a^*) \\
 &\geq \frac{\epsilon}{M} \sum_{m=1}^M q_{\pi}(s, a^m) + (1 - \epsilon) \sum_{m=1}^M \frac{\pi(a^m|s) - \epsilon/M}{1 - \epsilon} q_{\pi}(s, a^m) \\
 &= \sum_{m=1}^M \pi(a^m|s) q_{\pi}(s, a^m) = v_{\pi}(s)
 \end{aligned}$$

ϵ -Greedy Improvement Theorem

Let π be computed from an action-value function using ϵ -greedy algorithm. Assume $\bar{\pi}$ is derived by ϵ -greedy improvement from $q_{\pi}(s, a)$; then, $\bar{\pi} \geq \pi$

Online Control Loop via GPI and ϵ -Greedy Improvement

We can now build our control loop via ϵ -greedy algorithm

```
X_Control():
```

```
1: Initialize two random policies  $\pi$  and  $\bar{\pi}$ 
```

```
2: while  $\pi \neq \bar{\pi}$  do
```

```
3:    $\hat{q}_\pi = \text{X\_QUpdate}(\pi)$  and  $\pi \leftarrow \bar{\pi}$ 
```

```
4:    $\bar{\pi} = \epsilon\text{-Greedy}(\hat{q}_\pi)$ 
```

```
5: end while
```

Attention

We are still using *single update* of action-values for *policy improvement*: this means that we may have *bad* estimates of action-values at initial iterations!

First Try: Monte-Carlo Control Loop

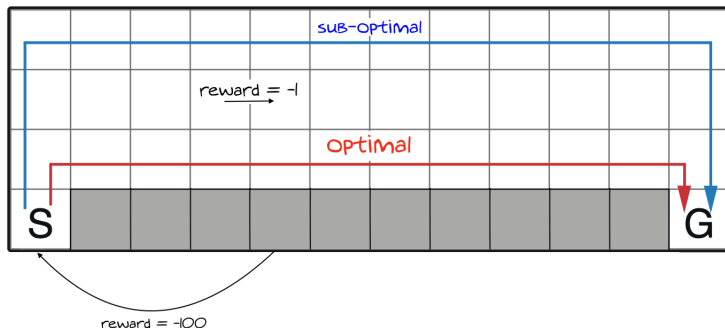
MC_Control():

- 1: Initialize estimator as $\hat{q}_\pi(s, a) = 0$ for *all states* and *actions*
- 2: **for** episode = 1 : K or until π stops changing **do**
- 3: Initialize with a *random state-action pair* (S_0, A_0)
- 4: Act via $\pi = \epsilon$ -Greedy($\hat{q}_\pi$)
- 5: Sample a trajectory

$$S_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \dots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T: \text{terminal}$$

- 6: Initialize with $G = 0$
- 7: **for** $t = T - 1 : 0$ **do**
- 8: Update current return $G \leftarrow R_{t+1} + \gamma G$
- 9: Update $\hat{q}_\pi(S_t, A_t) \leftarrow \hat{q}_\pi(S_t, A_t) + \alpha(G - \hat{q}_\pi(S_t, A_t))$
- 10: **end for**
- 11: **end for**

Example: Cliff Walking

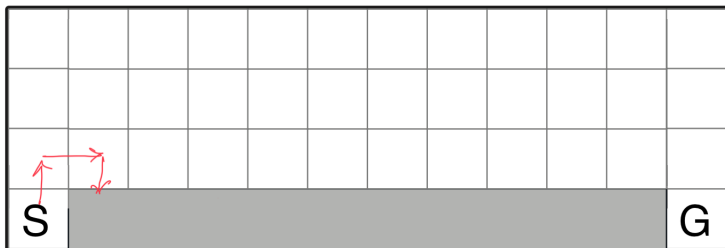


We have seen the cliff walking example in Assignment 1: we want to

- get from S to G with **shortest possible path**
- avoid hitting the cliff $\equiv$ gray squares
 - ↳ each time we hit the cliff, we get back to S with a **big negative reward**

Example: Cliff Walking

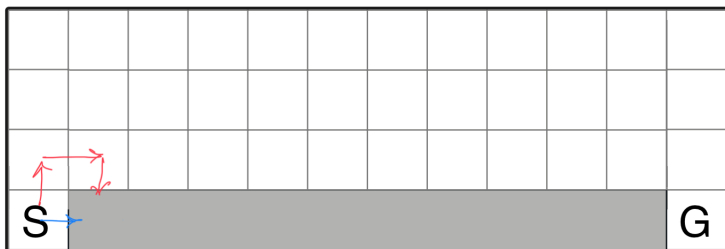
Say we use naive greedy policy: we **start** sampling trajectory and hit the cliff



We realize that our first action gave **bad** reward

Example: Cliff Walking

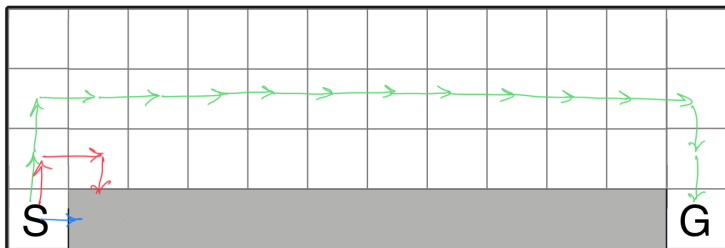
We now follow a *better* action, but we hit the cliff again



We realize that this action was even *worse*

Example: Cliff Walking

We get back to our first action, but now modify *next actions*



Say we are lucky and arrive at G

We will *never* go back to find the *optimal path*!

But with ϵ -greedy, we get the chance to *explore* again and find *optimal path*!

Greedy in Limit with Infinite Exploration Algorithms

- + Sounds working! But can we guarantee that this approach will **converge** to **optimal path**?
- Under some circumstances: **Yes!**

Recall that we said the following when we started Monte-Carlo

Asymptotic Convergence of Monte-Carlo

Let $C_K(s, a)$ denote number of visits at **state** s followed by **action** a during K Monte-Carlo episodes. Assume random initialization is distributed such that

$$\lim_{K \rightarrow \infty} C_K(s, a) = \infty$$

for any **state** s and **action** a ; then, we can guarantee $\hat{q}_\pi(s, a) \xrightarrow{K \uparrow \infty} q_\pi(s, a)$

Greedy in Limit with Infinite Exploration Algorithms

The main idea in this result was that

As long as we do *enough sampling*, so that we see *all states* and *actions* *enough* number of times, Monte-Carlo will *converge*

We can claim the same thing here

If we keep *playing enough*, we explore *all states* and *actions*; then, eventually we get *very sure* about *optimal values* and *actions*

But there is a small point here: if we keep on using ϵ -greedy policy even after we *got sure*, we can still perform sub-optimal

We should *stop exploring* once we have visited *all states* and *actions*

This is what we call

Greedy in Limit with Infinite Exploration $\equiv$ **GLIE**

GLIE Algorithms

GLIE Algorithms

A GPI-type control loop is GLIE, if for any **state-action** pair (s, a) , we have the following asymptotic properties

- 1 The number of visits to all **state-action** pair grows large

$$\lim_{K \rightarrow \infty} C_K(s, a) = \infty$$

- 2 The **improved** policy in last episode converges to **greedy** policy

$$\lim_{K \rightarrow \infty} \pi_K(a^m | s) = \begin{cases} 1 & m = \underset{m}{\operatorname{argmax}} q_{\pi_K}(s, a^m) \\ 0 & m \neq \underset{m}{\operatorname{argmax}} q_{\pi_K}(s, a^m) \end{cases}$$

GLIE control algorithms **converge** to **optimal policy**

GLIE Algorithms

- + It seems that they **contradict!** First one needs us to **explore** and the second to **exploit!**
- We could simply get rid of it by scaling ϵ

Say we choose ϵ to scale reversely by the number of episodes, e.g.,

$$\epsilon_k = \frac{1}{k}$$

Then, we have **both constraints** satisfied

- 1 We keep **exploring** a lot in **initial episodes**
- 2 We focus more on **exploiting** in **later episodes**

This is what we do in practice!

ϵ -Greedy Monte-Carlo is GLIE

MC_Control():

- 1: Initialize estimator as $\hat{q}_\pi(s, a) = 0$ for *all states* and *actions*
- 2: **for** episode = 1 : K or until π stops changing **do**
- 3: Initialize with a *random state-action pair* (S_0, A_0)
- 4: Set $\epsilon = 1/k$ and act via $\pi = \epsilon$ -Greedy($\hat{q}_\pi$)
- 5: Sample a trajectory

$$S_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \dots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T: \text{terminal}$$

- 6: Initialize with $G = 0$
- 7: **for** $t = T - 1 : 0$ **do**
- 8: Update current return $G \leftarrow R_{t+1} + \gamma G$
- 9: Update $\hat{q}_\pi(S_t, A_t) \leftarrow \hat{q}_\pi(S_t, A_t) + \alpha(G - \hat{q}_\pi(S_t, A_t))$
- 10: **end for**
- 11: **end for**